

# Church Numerals in Scheme (SICP Style)

## Definition

The Church numerals are a way of representing natural numbers using pure functions in the lambda calculus. Each number is represented as a higher-order function encoding the number of times a function is applied.

$$\begin{aligned}0 &\equiv \lambda f. \lambda x. x \\1 &\equiv \lambda f. \lambda x. f(x) \\2 &\equiv \lambda f. \lambda x. f(f(x)) \\3 &\equiv \lambda f. \lambda x. f(f(f(x))) \\&\vdots\end{aligned}$$

## Scheme Representation

The following code expresses Church numerals using closures in **Scheme**, consistent with the SICP approach.

```
#lang sicp

(define zero (lambda () '()))

(define (succ x) (lambda () x))
(define (pred x) (x))

(define one (succ zero))
(define two (succ one))
(define three (succ two))
(define four (succ three))
(define five (succ four))

(define (is-zero? x) (null? (x)))

(define (is-equal? x y)
  (cond ((is-zero? x) (is-zero? y))
        ((is-zero? y) (is-zero? x))
        (else (is-equal? (pred x) (pred y)))))

(define (add-church x y)
  (if (is-zero? y)
      x
      (add-church (succ x) (pred y))))
```

## Explanation

- **zero**: represented by a thunk returning the empty list `'()`.

- **succ**: successor function, which wraps a Church numeral in another closure.
- **pred**: predecessor, obtained by invoking the stored closure.
- **is-zero?**: checks if a Church numeral reduces to the base case '().
- **is-equal?**: recursively compares two numerals by unwrapping both.
- **add-church**: defines addition recursively using successor and predecessor.

## Concept

Each numeral here is a closure that captures its predecessor, forming a linked structure of thunks. The recursion in **add-church** effectively counts the applications of **succ**.

$$\text{add-church}(x, y) = \begin{cases} x & \text{if } y = 0 \\ \text{add-church}(\text{succ}(x), \text{pred}(y)) & \text{otherwise} \end{cases}$$

Thus, this representation realizes Peano arithmetic using pure lambda functions.